Resumen-DataDrivenDiffEq.jl

Documentation notation:

Type (generic, the most common placeholder)
Value
Vector-like
Proximal operator
Parameter
Number or integer
Function
State
Size
Real number

A<:B means that A is a (sub)type of B.

`T`, `V`, `P`, `R` are **type parameter names**, not **field names**. The actual field names are defined *inside* the struct body, something like:

```
mutable struct SR3{T, V, P<:AbstractProximalOperator}
    threshold::T
    lambda::V
    proximal::P
end
```

And how to read the fields of the struct? This way:

```
fieldnames(X)
```

Problems

`DataDrivenDiffEq.DataDrivenProblem` — Type

struct DataDrivenProblem{dType, cType, probType} <: DataDrivenDiffEq.AbstractDataDrivenProblem{dType, cType, probType}

The `DataDrivenProblem` defines a general estimation problem given measurements, inputs and (in the near future) observations. Three construction methods are available:

- `DirectDataDrivenProblem` for direct mappings.
- `DiscreteDataDrivenProblem` for time discrete systems.
- `ContinuousDataDrivenProblem` for systems continuous in time.
- `X` : State measurements

- `t` : Time measurements (optional)
- `DX` : Differential state measurements (optional); Used for time continuous problems
- `Y` : Output measurements (optional); Used for direct problems
- `U` : Input measurements (optional); Used for non-autonomous problems
- `p` : Parameters associated with the problem (optional)
- `name` : Name of the problem

The key invariant that makes it *continuous* (as opposed to discrete) is that **`X` and `DX` must be defined at the same time points**. The library encodes this in `probType` as a type tag — it doesn't store extra data, it just changes how downstream solvers interpret the problem.

If you pass `t` instead of `DX`, the library estimates derivatives numerically — which for noisy paleoclimate data is a critical and potentially problematic step worth paying attention to.

```
prob = DiscreteDataDrivenProblem(X)
prob = ContinuousDataDrivenProblem(X, DX)
prob = ContinuousDataDrivenProblem(X, t)
input_signal(u,p,t) = t^2
prob = DiscreteDataDrivenProblem(X, t, input_signal)

problem = DiscreteDataDrivenProblem(X)
problem = DiscreteDataDrivenProblem(X, t)
problem = DiscreteDataDrivenProblem(X, t, U)
problem = DiscreteDataDrivenProblem(X, t, U, p = p)
problem = DiscreteDataDrivenProblem(X, t, (x, p, t) -> u(x, p, t))

# Using available data
problem = ContinuousDataDrivenProblem(X, DX)
problem = ContinuousDataDrivenProblem(X, t, DX)
problem = ContinuousDataDrivenProblem(X, t, DX, U, p = p)
problem = ContinuousDataDrivenProblem(X, t, DX, (x, p, t) -> u(x, p, t))

# Using collocation
problem = ContinuousDataDrivenProblem(X, t, InterpolationMethod())
problem = ContinuousDataDrivenProblem(X, t, GaussianKernel())
problem = ContinuousDataDrivenProblem(X, t, U, InterpolationMethod())
problem = ContinuousDataDrivenProblem(X, t, U, GaussianKernel(), p = p)

problem = DirectDataDrivenProblem(X, Y)
problem = DirectDataDrivenProblem(X, t, Y)
problem = DirectDataDrivenProblem(X, t, Y, U)
problem = DirectDataDrivenProblem(X, t, Y, p = p)
problem = DirectDataDrivenProblem(X, t, Y, (x, p, t) -> u(x, p, t), p = p)
```

`DataDrivenDiffEq.DiscreteDataDrivenProblem` — Function

A time discrete `DataDrivenProblem` useable for problems of the form `f(x[i],p,t,u) ↦ x[i+1]`.

`DataDrivenDiffEq.ContinuousDataDrivenProblem` — Function

A time continuous `DataDrivenProblem` useable for problems of the form `f(x,p,t,u) ↦ dx/dt`.

This is the one I will be using for dynamics discovery!

`DataDrivenDiffEq.DirectDataDrivenProblem` — Function

A direct `DataDrivenProblem` useable for problems of the form `f(x,p,t,u) ↦ y`.

`DataDrivenDiffEq.DataDrivenDataset` — Type

struct DataDrivenDataset{N, U, C} <: DataDrivenDiffEq.AbstractDataDrivenProblem{N, U, C}

A collection of DataDrivenProblems used to concatenate different trajectories or experiments. Holds multiple trajectories from the **same system**, e.g. different initial conditions. It's for aggregating data before solving, not for bootstrapping.

It has three constructors available:

- `DirectDataset` for direct problems
- `DiscreteDataset` for discrete problems
- `ContinuousDataset` for continuous problems

Fields:

- `name`: Name of the dataset
- `probs`: The problems
- `sizes`: The length of each problem - for internal use

Ensemble SINDy is controlled through `DataDrivenCommonOptions`, specifically the `sampler` and `ensemble` keywords. The pattern looks like:

julia

```
options = DataDrivenCommonOptions(
    sampler = DataSampler(Batcher(n = 100)),  # bootstrap sampler
    ensemble = true
)
ddsol = solve(ddprob, basis, opt, options = options)
```

`DataDrivenDataset` is for when you have **multiple trajectories** of the same system — different initial conditions, different experiments, different time windows — and you want to fit a **single** `f` that explains all of them simultaneously.

For your thesis, a concrete use case would be: if you have **multiple glacial cycles** as separate segments of your proxy record, you could package each cycle as a separate trajectory in a `DataDrivenDataset`, then solve once. This forces SINDy to find a `f` that is consistent across all cycles, which is more robust than fitting to a single long trajectory.

`DataDrivenDiffEq.DiscreteDataset` **— Function**

`DataDrivenDiffEq.ContinuousDataset` **— Function**

`DataDrivenDiffEq.DirectDataset` **— Function**

Basis

`DataDrivenDiffEq.Basis` **— Type**

struct Basis{IMPL, CTRLS} <: DataDrivenDiffEq.AbstractBasis

A basis over the states with parameters, independent variable, and possible exogenous controls. It extends an `AbstractSystem` as defined in `ModelingToolkit.jl`. `f` can either be a Julia function which is able to use ModelingToolkit variables or a vector of `eqs`. It can be called with the typical SciML signature, meaning out of place with `f(u,p,t)` or in place with `f(du, u, p, t)`. If control inputs are present, it is assumed that no control corresponds to zero for all inputs. The corresponding function calls are `f(u,p,t,inputs)` and `f(du,u,p,t,inputs)` and need to be specified fully.

If `linear_independent` is set to `true`, a linear independent basis is created from all atom functions in `f`.

If `simplify_eqs` is set to `true`, `simplify` is called on `f`.

Additional keyword arguments include `name`, which can be used to name the basis, and `observed` for defining observables.

Fields

- `eqs`: The equations of the basis
- `unknowns`: Dependent (state) variables
- `ctrls`: Control variables
- `ps`: Parameters
- `observed`: Observed
- `iv`: Independent variable
- `implicit`: Implicit variables of the basis
- `f`: Internal function representation of the basis
- `name`: Name of the basis
- `systems`: Internal systems

Example:

```
@parameters t
@variables V, T, C, I

Ψ = Basis([V; T; C; V^2; T*V; cos(2π*I)], [V, T, C], iv = t)
```

Where:

- `V` — ice volume
- `T` — deep ocean temperature
- `C` — CO_2
- `I` — insolation (goes in `U`, not in the state, since it's a **known external forcing**)

The candidate functions you choose encode your **prior physical knowledge**:

- `[V, T, C]` — linear interactions
- `V^2, T*V, ...` — nonlinear couplings
- `cos(2π*I)` — periodic response to orbital forcing
-

The critical scientific decision is: **what do you put in this library?** SINDy can only find terms you include. If the true `f` contains `V³` and you didn't include it, it won't appear.

API

`DataDrivenDiffEq.dynamics` — Function
`DataDrivenDiffEq.jacobian` — Function
`DataDrivenDiffEq.get_parameter_values` — Function
`DataDrivenDiffEq.get_parameter_map` — Function

Generators

`DataDrivenDiffEq.polynomial_basis` — Function

Solutions

`DataDrivenDiffEq.DataDrivenSolution` — Type

struct DataDrivenSolution{T} <: DataDrivenDiffEq.AbstractDataDrivenSolution

The solution to a `DataDrivenProblem` derived via a certain algorithm. The solution is represented via an `Basis`, which makes it callable.

Fields

- `basis`: The basis representation of the solution
- `retcode`: Returncode
- `alg`: Algorithm

- `out` : Original output of the solution algorithm
- `prob` : Problem
- `residuals` : Residual sum of squares
- `dof` : Degrees of freedom
- `internal_problem` : Internal problem

`DataDrivenDiffEq.get_problem` — Function
`DataDrivenDiffEq.get_basis` — Function
`DataDrivenDiffEq.get_algorithm` — Function
`DataDrivenDiffEq.get_results` — Function
`DataDrivenDiffEq.is_converged` — Function
`StatsAPI.dof` — Function
`StatsAPI.rss` — Function
`StatsAPI.nobs` — Function
`StatsAPI.loglikelihood` — Function
`StatsAPI.nullloglikelihood` — Function
`StatsAPI.r2` — Function
`StatsBase.summarystats` — Function

Utilities

DataDrivenSparse.jl (optimizators and parse thresholds)

Fundamentals

> DataDrivenSparse provides a universal framework to infer system of equations using sparse regression. Assume the system:

$$y_i = f(x_i, p, t_i, u_i)$$

> Then might be able to express the unknown function ff as a linear combination of basis elements φi:
>
> $$y_i = \sum_{j=i}^{k} \xi_i \phi_k(x_i, p, t_i, u_i)$$
>
> And then solve the least squares problem (but with sparsity) **Revise, it's not complete**:

$$\Xi'_{s.t.} = \min_{\Xi} ||\Xi||_0$$

> Implicit y can be solved via ImplicitOptimizer function.

Optimizers

`DataDrivenSparse.STLSQ` **— Type**

Implements the Brunton 2016 algorithm.

struct STLSQ{T<:Union{Number, AbstractVector}, R<:Number} <: DataDrivenSparse.AbstractSparseRegressionAlgorithm

> `STLSQ` is a **generic struct** that implements the sparse regression algorithm interface. It's parameterized so it can flexibly accept either a single threshold number or a vector of thresholds, plus a numeric regularization parameter.

```julia
opt = STLSQ()
opt = STLSQ(1e-1)
opt = STLSQ(1e-1, 1.0) # Set rho to 1.0
opt = STLSQ(Float32[1e-2; 1e-1])
```

`DataDrivenSparse.ADMM` — Type

ADMM is an implementation of Lasso using the alternating direction methods of multipliers, and loosely based on https://web.stanford.edu/~boyd/papers/admm/lasso/lasso.html.

mutable struct ADMM{T(any), R<:Number} <: DataDrivenSparse.AbstractSparseRegressionAlgorithm

```
    opt = ADMM()
    opt = ADMM(1e-1, 1.0)
```

`DataDrivenSparse.SR3` — Type

`SR3` is an optimizer framework introduced by Zheng et al., 2018 and used within Champion et al., 2019. `SR3` contains a sparsification parameter `λ`, a relaxation `ν`.

mutable struct SR3{T, V, P<:DataDrivenSparse.AbstractProximalOperator} <: DataDrivenSparse.AbstractSparseRegressionAlgorithm

```
opt = SR3()
opt = SR3(1e-2)
opt = SR3(1e-3, 1.0)
opt = SR3(1e-3, 1.0, SoftThreshold())
```

`DataDrivenSparse.ImplicitOptimizer` — Type

(It exists)

Proximal operators

Think of them as **thresholding rules** — they decide which coefficients survive and which get zeroed out during sparse regression.

`DataDrivenSparse.SoftThreshold` — Type

struct SoftThreshold <: DataDrivenSparse.AbstractProximalOperator

Implements Zheng et al., 2018.

The classic.

Small values get pushed to exactly 0
Large values get shrunk by λ
Equivalent to **LASSO** regularization

`DataDrivenSparse.HardThreshold` — Type

struct HardThreshold <: DataDrivenSparse.AbstractProximalOperator

- No shrinkage, just elimination
- Sparser results than soft thresholding

Implements Zheng et al., 2018.

`DataDrivenSparse.ClippedAbsoluteDeviation` — Type

struct ClippedAbsoluteDeviation{T} <: DataDrivenSparse.AbstractProximalOperator

- A **hybrid**: acts like soft threshold for small values, but leaves large values **unpenalized**
- Reduces bias on large coefficients (a known problem with pure LASSO)

`T` : Upper threshold

```
opt = ClippedAbsoluteDeviation()
opt = ClippedAbsoluteDeviation(1e-1)
```

DataDrivenSR.jl (symbolic regression wrapp)

> DataDrivenSR provides an API to `SymbolicRegression.jl` to infer equations as:

$$y_i = f(x_i, p, t_i, u_i)$$

So it's essentially a **wrapper** — it connects the DataDrivenDiffEq ecosystem to the power of SymbolicRegression.jl.

`DataDrivenSR.EQSearch` — Type

struct EQSearch <: DataDrivenDiffEq.AbstractDataDrivenAlgorithm

Options for using SymbolicRegression.jl within the `solve` function. Automatically creates `Options` with the given specification. Sorts the operators stored in `functions` into unary and binary operators on conversion.

Fields

- `weights` : Optionally weight the loss for each y by this value (same shape as y) Default: nothing
- `numprocs` : The number of processes to use, if you want `equation_search` to set this up automatically. Default: nothing
- `procs` : If you have set up a distributed run manually with procs = addprocs() and @everywhere, pass the procs to this keyword argument. Default: nothing
- `addprocs_function` : If using multiprocessing (parallelism=:multithreading), and are not passing procs manually, then they will be allocated dynamically using addprocs. However, you may also pass a custom function to use instead of addprocs. This function should take a single positional argument, which is the number of processes to use, as well as the lazy keyword argument. For example, if set up on a slurm cluster, you could pass addprocs_function = addprocs_slurm, which will set up slurm processes. Default: nothing
- `parallelism` : What parallelism mode to use. The options are :multithreading, :multiprocessing, and :serial. Multithreading uses less memory, but multiprocessing can handle multi-node compute. If using :multithreading mode, the number of threads available to Julia are used. If using :multiprocessing, numprocs processes will be created dynamically if procs is unset. If you have already allocated processes, pass them to the procs argument, and they will be used. You may also pass a string instead of a symbol. Default: :serial
- `runtests` : Whether to run (quick) tests before starting the search, to see if there will be any problems during the equation search related to the host environment Default: true
- `eq_options` : Options for `equation_search` Default: SymbolicRegression.Options()

Example

```
using DataDrivenDiffEq
using LinearAlgebra
using OrdinaryDiffEq
using DataDrivenSR

A = [-0.9 0.2; 0.0 -0.5]
B = [0.0; 1.0]
u0 = [10.0; -10.0]
tspan = (0.0, 20.0)

f(u, p, t) = A * u .+ B .* sin(0.5 * t)
```

```
sys = ODEProblem(f, u0, tspan)
sol = solve(sys, Tsit5(), saveat = 0.01);

X = Array(sol)
t = sol.t
U = permutedims(sin.(0.5 * t))
prob = ContinuousDataDrivenProblem(X, t, U = U)

eqsearch_options = SymbolicRegression.Options(binary_operators = [+, *],
    loss = L1DistLoss(),
    verbosity = -1, progress = false, npop = 30,
    timeout_in_seconds = 60.0)

alg = EQSearch(eq_options = eqsearch_options)

res = solve(prob, alg, options = DataDrivenCommonOptions(maxiters = 100))

loglikelihood(res)

system = get_basis(res)
```

DataDrivenDMD.jl

NOT IMPORTANT

Personal insights

1. Insolation and other time-dependent (maybe CO2) parameters must go as control parameters, U. Internal parameters, like Volume must be in the state part, X. That is to account that the library solves autonomous dynamical systems, but with control parameters that can change in time (so it solves this way a non-autonomous system).
2. Think if I isolate the different glacial cycles and adjust them via a dataset. If I isolate glaciation and deglaciation and fit them via two different datasets. Or if I just deliver the full timeseries and I obtain the full function. (How do I isolate?)
3. I need to go to PACCO and search for the independent variables (and it's functional form), to form a basis with that, not with all the variables (or SINDy would break with that much correlations).
4. Solutions also offers ways to determine the robusticity. For example, the r^2. So maybe I don't need the ensemble thing.